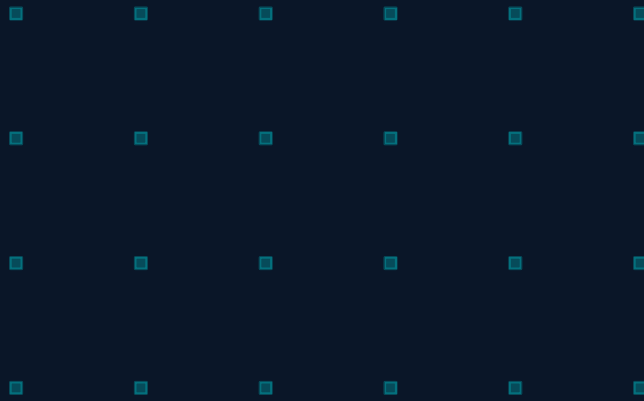


13th LISA Cosmology Working Group Workshop

June 1–5, 2026 · Institute of Cosmos Sciences (ICC), University of Barcelona



Why Do We Use

Slurm?

From chaos to order on shared compute clusters

Jam Sadiq
June 4, 2026

A story about 50 researchers and one cluster



Monday, 9 AM.

50 researchers. One cluster. No rules.

Alice

SSHs into node01
starts a 32-core job.
Node is already at 100%.

Bob

Picks node02.
His job crashes — someone
else filled the RAM.

Carlos

Gives up. Runs job on
his laptop instead.
Takes 3 days.

This was real life on clusters before job schedulers.

Enter Slurm

Simple Linux Utility for Resource Management



Queues jobs

Everyone submits — Slurm decides who runs when



Allocates fairly

Based on priority, partition limits, past usage



Runs your script

On the exact CPUs/GPUs/RAM you requested



Tracks everything

Job history, user accounting, failures

Think of Slurm as...

An airport control tower

You file a flight plan (job script). The tower decides your runway (node), takeoff time (queue), and tells you exactly how long you can be in the air (time limit).

No tower = planes colliding.

Slurm basics

a common job scheduler — the commands you will use every day

Submit a job

sbatch

Submit a script — runs later when resources free

srun

Run a command interactively on a compute node

salloc

Reserve resources now, run commands manually inside

Monitor & cancel

squeue

See jobs in the queue

sinfo

See node/partition availability

sacct

Job accounting & history

scancel

Cancel a running or pending job

Key flags — works with sbatch / srun / salloc

-c <N>

CPUs

e.g. **-c 4**

4 CPU cores

--mem-per-cpu <MEM>

Memory

e.g. **--mem-per-cpu 1G**

1 GB × CPUs → 4 CPUs = 4 GB total

-t <TIME>

Time

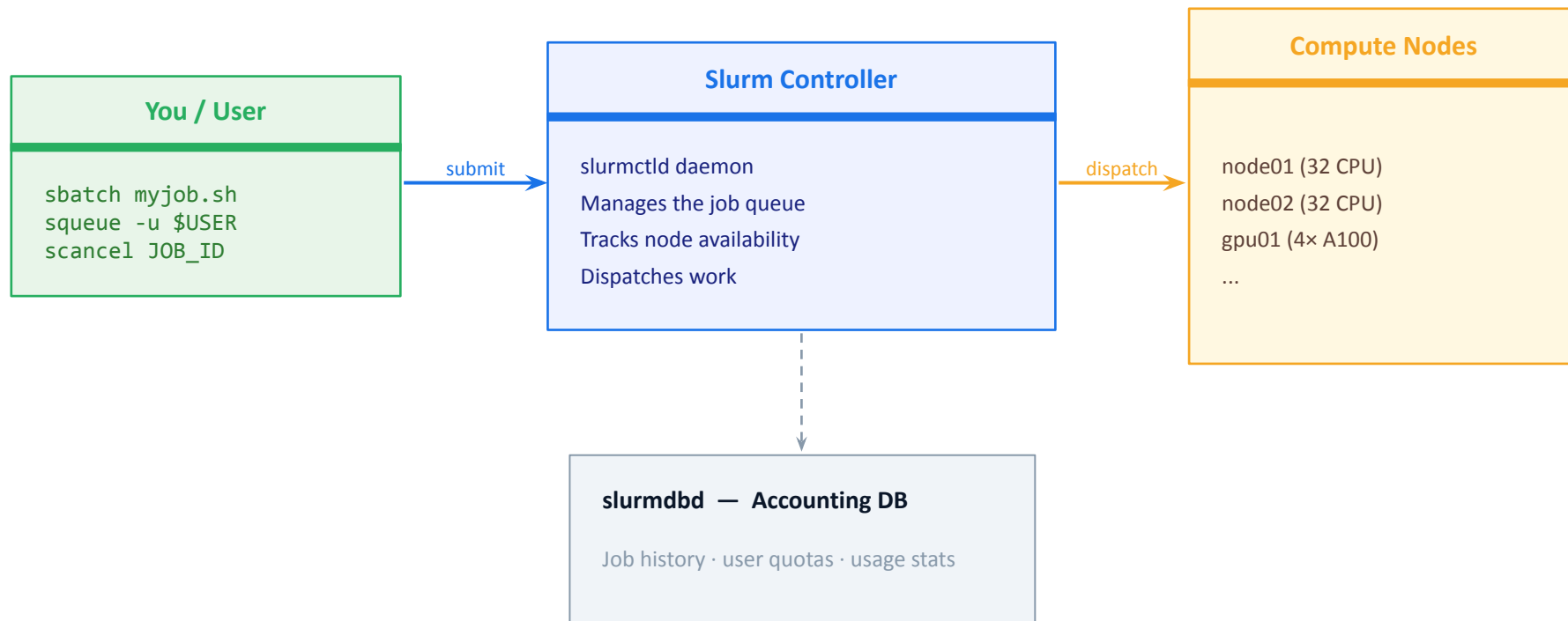
e.g. **-t 5:0:0**

5 hours | 10:0 = 10 min | 3-0:0:0 = 3 days

Example

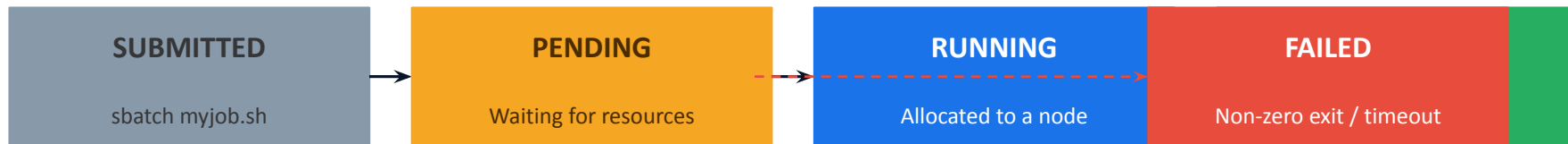
```
sbatch -c 4 --mem-per-cpu 1G -t 5:0:0  
job.sh
```

The Three Pieces



Each compute node runs slurmd daemon — reports back to controller

What Happens to Your Job



Slurm injects these environment variables into your running script:

<code>\$SLURM_JOB_ID</code>	Unique ID of your job	<code>\$SLURM_CPUS_PER_TASK</code>	How many cores you got
<code>\$SLURMD_NODENAME</code>	Which node you're on	<code>\$SLURM_ARRAY_TASK_ID</code>	Your index in a job array

Your First Job Script

myjob.sh

```
#!/bin/bash

#SBATCH --job-name=hello_slurm
#SBATCH --output=hello_%j.out
#SBATCH --error=hello_%j.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=4G
#SBATCH --time=00:10:00

echo "Job ID: $SLURM_JOB_ID"
echo "Node:  $SLURMD_NODENAME"
echo "CPUs:  $SLURM_CPUS_PER_TASK"

python my_script.py
```

directives — all the #SBATCH lines

resources — cores, memory, time

your actual work

Live Demo — 5 Commands

Run these in order on the cluster

1

```
sbatch myjob.sh
```

Submit the job — note the job ID printed back

2

```
squeue -u $USER
```

Watch your job: ST column shows PD (pending) → R (running)

3

```
scontrol show job <ID>
```

Detailed view: node assignment, start time, memory used

4

```
cat hello_<ID>.out
```

See your output file once the job finishes

5

```
sacct -j <ID>
```

Accounting: actual runtime, exit code, CPU efficiency

Pro tip: use `watch -n 2 squeue -u $USER` to auto-refresh every 2 seconds

Key #SBATCH Directives

Directive	What it does
<code>--ntasks</code>	Number of parallel processes (MPI tasks)
<code>--cpus-per-task</code>	Cores per process — use for multithreaded code
<code>--mem</code>	Total RAM e.g. 8G, 500M
<code>--time</code>	Wall-clock limit HH:MM:SS — job is killed if exceeded
<code>--partition</code>	Which queue e.g. gpu, short, long, highmem
<code>--gres=gpu:1</code>	Request 1 GPU (any type)
<code>--array=1-50</code>	Launch 50 jobs, each gets SLURM_ARRAY_TASK_ID
<code>--output=%j.out</code>	%j = job ID, %x = job name, %a = array index

Best Practices

✓ DO

Always set `--time` (estimate + 20%)

Request only what you actually need

Test interactively first with `srunch --pty bash`

Use job arrays for parameter sweeps

Read `.out` and `.err` logs when jobs fail

✗ DON'T

Run heavy work on the login node

Request 32 CPUs when you use 4

Submit 100 separate jobs for sweep parameters

Hardcode node names in your script (`ssh node07 python script.py`)

Forget to set `--mem` (may get killed)

Leave zombie jobs running — use `scancel`

Learn More

Resources to bookmark after this talk

Official Slurm docs

Reference for every directive and command

<https://slurm.schedmd.com/documentation.html>

Slurm quick-start guide

Setup and first jobs walkthrough

<https://slurm.schedmd.com/quickstart.html>

NERSC Slurm tutorial

Excellent real-world examples from a top HPC centre

<https://docs.nersc.gov/jobs/>

Cheat sheet (HPC @ NIH)

Commands at a glance for quick reference

<https://hpc.nih.gov/docs/userguide.html>

Interactive jobs tutorial

srun reference — test before you batch

<https://slurm.schedmd.com/srun.html>

Slurm intro video (YouTube)

Search 'slurm job scheduler tutorial' for video walkthroughs

https://www.youtube.com/results?search_query=slurm+job+scheduler+tutorial

■ ■ ■ ■ ■ ■

Remember the story.

■ ■ ■ ■ ■ ■

■ ■ ■ ■ ■ ■

Alice, Bob, Carlos

couldn't share a cluster without chaos.

■ ■ ■ ■ ■ ■

Slurm is the solution —

a fair, automated traffic controller

for your compute resources.

sbatch

squeue

scancel

sacct

srun

--array: run one script many times

for parameter sweeps, input files, random seeds — any embarrassingly parallel work

✗ Without arrays — the painful way

```
sbatch train.sh --lr 0.1
sbatch train.sh --lr 0.01
sbatch train.sh --lr 0.001
sbatch train.sh --lr 0.0001
sbatch train.sh --lr 0.00001
```



5 separate job IDs to track



Need all 5 IDs just to cancel



500 runs? Totally unmanageable

✓ With arrays — one sbatch, many tasks

```
#SBATCH --array=0-4

LR_VALUES=(0.1 0.01 0.001
           0.0001 0.00001)

LR=${LR_VALUES[$SLURM_ARRAY_TASK_ID]}

python train.sh --lr $LR
```



One job ID e.g. 42601 — tasks become 42601_1 ... 42601_5



scancel 42601 cancels ALL at once



Add %20 to run max 20 at a time: --array=0-499%20

`$SLURM_ARRAY_TASK_ID` is just a number Slurm injects — use it to index any list of parameters, filenames, or configs

Job array — full working example

learning rate sweep: 5 values, one sbatch

train_sweep.sh

```
#!/bin/bash
#SBATCH --job-name=lr_sweep
#SBATCH --array=0-4
#SBATCH --cpus-per-task=4
#SBATCH --mem-per-cpu=2G
#SBATCH --time=2:0:0
#SBATCH --output=logs/run_%a.out

# parameter list
LR_VALUES=(0.1 0.01 0.001 0.0001 0.00001)

# pick value for THIS task
LR=${LR_VALUES[$SLURM_ARRAY_TASK_ID]}

echo "Task $SLURM_ARRAY_TASK_ID lr=$LR"
python train.py --lr $LR \
  --out results/model_${SLURM_ARRAY_TASK_ID}
```

--array=0-4

Creates tasks _0 _1 _2 _3 _4 under one job ID

%a in output filename

Each task writes its own log: run_0.out ... run_4.out

\$SLURM_ARRAY_TASK_ID

The key — Slurm sets this to 0,1,2,3,4 in each task. You use it to pick the right parameter.

What Slurm runs

42601_0	LR = 0.1
42601_1	LR = 0.01
42601_2	LR = 0.001
42601_3	LR = 0.0001

Throttle tip: `--array=0-499%20` runs at most 20 tasks at once — keeps the cluster fair for everyone